

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

9. let, where, λ

let y where

- Introducen definiciones auxiliares *locales*.

```
impares1 :: Int -> [Int]
impares1 n = map f [1..n]
            where f x = 2*x-1
```

```
impares2 :: Int -> [Int]
impares2 n = let f x = 2*x-1
            in map f [1..n]
```

- Puede haber más de una definición.
- Se pueden definir valores, no solo funciones.

```
impares3 :: Int -> [Int]
impares3 n = map f lista
            where f x = 2*x-1
                  lista = [1..n]
```

- **let** ... **in** ... es una expresión:
 - `4 * (let x = 9 in x + 1) + 2`
 - Esto no se puede hacer con **where**.

```
zona :: (Double, Double) -> String
zona (x, y) | r < 1      = "dentro"
            | r == 1     = "en el borde"
            | otherwise = "fuera"
            where r = sqrt (x*x + y*y)
```

- Las definiciones del **where** alcanzan a todas las guardas de la ecuación.
 - Esto no se puede hacer con **let**.

λ -expresiones

- Una λ -expresión es una forma de definir una función sin darle nombre.
 - En matemáticas: $x \mapsto 2x - 1$.
 - En λ -cálculo: $\lambda x.2x - 1$.
 - En Haskell: `\x -> 2*x-1`.

```
impares, impares', impares'' :: Int -> [Int]
impares n = map (\x -> 2*x-1) [1..n]
impares' n = let f = \x -> 2*x-1
              in map f [1..n]
impares'' = \n -> map (\x -> 2*x-1) [1..n]

sumar :: Int -> Int -> Int
sumar = \x y -> x + y

constante :: a -> b -> a
constante k = \_ -> k
```

- Se pueden usar patrones:

```
primer = \(x,y) -> x
sumas = map (\(a,b) -> a + b) [(1,2), (3,5), (6,3), (2,6)]
```

Ejercicios

1. ¿Cuál es el resultado (tipo y valor) de esta expresión?

```
| otroRes = (\f -> f 3) (\x -> x + 2)
```

2. ¿Cuál es el resultado (tipo y valor) de esta expresión?

```
| resultado = (\f -> \g x -> f (g (f x))) (\x -> 2*x) (\x -> x+1) 3
```

3. ¿Cuál es el tipo de esta expresión: `\x -> x 0 == 0`?

4. Define una función `raices :: Float -> Float -> Float -> [Float]` que devuelva en una lista las raíces reales de la ecuación $ax^2 + bx + c = 0$.

- `raices 1 (-3) 2` $\rightsquigarrow$ `[2.0, 1.0]`
- `raices 1 2 1` $\rightsquigarrow$ `[-1.0]`
- `raices 1 0 1` $\rightsquigarrow$ `[]`

El discriminante $b^2 - 4ac$ debe ser calculado solo una vez en cada llamada.

5. La derivada de una función f en un punto x se puede aproximar por el cociente

$$\frac{f(x+h) - f(x-h)}{2h}$$

con h pequeño. Define la función

```
deriva :: Float -> (Float -> Float) -> (Float -> Float)
```

de manera que `deriva h f` sea una aproximación a la función derivada de f , calculada con ese h . El cuerpo de `deriva` debe ser una λ -expresión: `deriva h f = \x -> ...`. Ejemplos:

- `deriva 0.001 (\x -> x*x) 3` $\rightsquigarrow$ `5.9995646`
- `map (deriva 0.001 (\x->x*x)) [1,2,3]` $\rightsquigarrow$ `[2.000004,3.9999483,5.9995646]`

La función `deriva h` toma una función y devuelve otra función: es una función *de orden superior*.